Universidade Estadual do Centro-Oeste
Departamento de Ciências da Computação
Gabriel Echeverria, Gabriel V. Pereira,
Nael B. Barreto, Victor L. P. da Silva

Algoritmos e Programação II
2º Semestre do 1º Ano
20 de Outubro de 2026

Relatório do Desafio em Algoritmos e Programação de Computadores II

Sumário

1 Introdução **2**
1.1 Contextualização . . . 2
1.2 Objetivos . . . 2
1.3 Organização do Relatório . . . 3

2 Modelagem & Arquitetura do Sistema **3**
2.1 Estruturas de Dados . . . 4
2.2 Armazenamento e Gerenciamento de Memória . . . 5
2.3 Modularização . . . 5
2.4 Cadastro e Validação de Dados . . . 6

3 Algoritmos de Ordenação **7**
3.1 InsertionSort . . . 7
3.2 MergeSort . . . 7
3.3 Ordenação Indireta . . . 8
3.4 Comparação Teórica . . . 8

4 Algoritmos de Busca **8**
4.1 Busca Sequencial . . . 8
4.2 Busca Binária . . . 9
4.3 Estratégias de Busca no Sistema . . . 9

5 Algoritmos Recursivos **10**
5.1 Modelagem das Relações entre Evidências . . . 10
5.2 Cálculo Recursivo da Relevância . . . 10
5.3 Identificação da Cadeia de Maior Relevância . . . 10
5.4 Análise de Complexidade . . . 11

6 Metodologia de Avaliação de Desempenho **11**
6.1 Ambiente de Testes . . . 11
6.2 Geração de Dados . . . 11
6.3 Métricas Utilizadas . . . 12
6.4 Procedimentos Experimentais . . . 12

7 Resultados e Discussão **12**
7.1 Desempenho dos Algoritmos de Ordenação 13
7.2 Desempenho dos Algoritmos de Busca 13
7.3 Comparação entre Complexidade Teórica e Experimental 14
7.4 Limitações 14

8 Conclusão **14**
8.1 Resultados Alcançados 14
8.2 Principais Conclusões 14
8.3 Melhorias Possíveis 14

1 Introdução

1.1 Contextualização

O gerenciamento de grandes volumes de informações representa um desafio significativo para sistemas computacionais, especialmente quando esses dados precisam ser frequentemente consultados, organizados e analisados. No contexto deste desafio, isso se manifesta na necessidade de administrar registros de ocorrências, pessoas envolvidas e evidências coletadas, estabelecendo relações entre essas informações e permitindo sua recuperação de maneira eficiente.

À medida que a quantidade de registros aumenta, a escolha das estruturas de dados e dos algoritmos utilizados passa a exercer influência direta sobre o desempenho do sistema. Operações como localizar uma ocorrência específica, ordenar indivíduos por seus atributos ou analisar evidências relacionadas podem apresentar custos computacionais consideravelmente diferentes, dependendo das estratégias empregadas.

1.2 Objetivos

O objetivo geral deste trabalho é desenvolver um sistema computacional para o cadastro, a organização, a consulta e a análise de informações relacionadas a ocorrências policiais, aplicando estruturas de dados e algoritmos estudados na disciplina de Algoritmos e Programação de Computadores II.

Como objetivos específicos, destacam-se:

- Implementar estruturas de dados para o armazenamento de informações sobre pessoas, ocorrências e evidências, utilizando a linguagem C++
- Desenvolver mecanismos de cadastro e validação que garantam a consistência dos registros e das relações entre eles

- Implementar e comparar os algoritmos de ordenação, utilizando diferentes critérios de ordenação
- Implementar os algoritmos de busca sequencial e binária, analisando suas características e requisitos de utilização
- Empregar funções recursivas para analisar relações entre evidências associadas a uma ocorrência
- Avaliar experimentalmente o desempenho dos algoritmos implementados, considerando o tempo de execução, o número de comparações e as movimentações realizadas
- Relacionar os resultados experimentais às respectivas complexidades computacionais teóricas, identificando vantagens e limitações de cada abordagem

1.3 Organização do Relatório

Este relatório está organizado em oito seções. Após esta introdução, a Seção 2 apresenta a modelagem dos dados e a arquitetura do sistema, incluindo as estruturas utilizadas, a organização do código e os mecanismos de armazenamento e cadastro.

As Seções 3 e 4 descrevem, respectivamente, os algoritmos de ordenação e de busca implementados, apresentando seus princípios de funcionamento, suas particularidades e suas complexidades computacionais. A Seção 5 aborda as funcionalidades recursivas desenvolvidas para a análise das relações entre evidências.

Na Seção 6, são apresentados os procedimentos metodológicos empregados na avaliação de desempenho dos algoritmos. A Seção 7 reúne os resultados experimentais e sua discussão, relacionando as medições obtidas ao comportamento esperado a partir da análise teórica. Por fim, a Seção 8 apresenta as conclusões do trabalho, destacando os principais resultados, as limitações identificadas e possíveis melhorias futuras.

2 Modelagem & Arquitetura do Sistema

O sistema foi desenvolvido na linguagem C++, utilizando uma arquitetura modular, na qual as diferentes funcionalidades são organizadas em arquivos específicos. A modelagem dos dados foi realizada por meio de structs, enquanto o armazenamento dos registros utiliza vertores alocados dinamicamente.

A arquitetura adotada busca manter a separação entre as informações armazenadas e os algoritmos responsáveis por seu processamento. Dessa forma, operações de cadastro, busca, ordenação e análise podem ser executadas sobre os mesmos conjuntos de dados sem a necessidade de modificar sua organização física.

2.1 Estruturas de Dados

A representação dos dados foi desenvolvida utilizando três estruturas principais: `Pessoa`, `Ocorrencia` e `Evidencia`. Cada uma armazena um conjunto de atributos correspondentes às entidades do sistema.

A estrutura `Pessoa` contém o identificador único do indivíduo, seu nome completo, idade e cidade. A estrutura `Ocorrencia`, por sua vez, armazena um identificador único, o tipo da ocorrência, o local, o ano e o nível de gravidade.

A estrutura `Evidencia` representa as informações coletadas durante uma investigação, contendo seu identificador, descrição, relevância e o identificador da ocorrência à qual pertence. Adicionalmente, foi introduzido o atributo `idEvidenciaPai`, que permite estabelecer relações hierárquicas entre diferentes evidências de uma mesma ocorrência.

A utilização desse atributo possibilita representar conjuntos de evidências por meio de árvores, nas quais uma evidência pode estar associada a várias evidências descendentes, mas possui, no máximo, uma evidência-pai. Evidências independentes são identificadas pelo valor zero no atributo `idEvidenciaPai`.

Além das estruturas principais, foi desenvolvida uma estrutura auxiliar denominada `Ordem`, responsável por armazenar índices utilizados nas operações de ordenação. Essa estrutura possui um ponteiro para um vetor de inteiros e um atributo que representa sua capacidade.

A estrutura `Ordem` permite ordenar logicamente os registros sem alterar suas posições originais na memória. Assim, diferentes critérios de ordenação podem ser aplicados sobre os índices, mantendo a integridade da organização dos registros principais.

Outra estrutura auxiliar, denominada `Metricas`, armazena contadores de comparações e movimentações realizadas pelos algoritmos. Seu objetivo é permitir a coleta de informações quantitativas para a avaliação de desempenho, apresentada nas seções posteriores.

2.2 Armazenamento e Gerenciamento de Memória

O armazenamento dos registros é realizado por meio de vetores de estruturas, alocados dinamicamente utilizando o operador new[] da linguagem C++. Essa abordagem permite determinar a capacidade dos conjuntos de dados durante a inicialização do programa, sem a necessidade de utilizar contêineres prontos da biblioteca padrão. Foram estabelecidas capacidades iniciais de 10.000 pessoas, 5.000 ocorrências e 20.000 evidências, conforme os requisitos mínimos propostos para o sistema.

Cada conjunto de registros possui um contador responsável por indicar a quantidade de elementos efetivamente cadastrados. Dessa maneira, existe uma distinção entre a capacidade total do vetor e o número de posições que contêm registros válidos. Antes de realizar um novo cadastro, o sistema verifica se ainda existe capacidade disponível.

Os identificadores dos registros são gerados sequencialmente, iniciando em 1. Como os registros são inseridos na próxima posição disponível dos vetores principais, estabelece-se uma correspondência direta entre o identificador e a posição do elemento: Indíce = ID - 1

Para preservar essa característica, as operações de ordenação utilizam vetores auxiliares de índices. Cada índice representa uma posição no vetor original e pode ser reorganizado independentemente dos registros armazenados. Dessa forma, o sistema permite diferentes visualizações ordenadas sem comprometer a organização utilizada pelas operações de acesso direto e busca.

A memória alocada dinamicamente é liberada por meio do operador delete[]. Os vetores principais são desalocados ao encerramento do programa, enquanto a estrutura Ordem possui um destrutor responsável pela liberação de seu vetor de índices. Estruturas auxiliares temporárias, utilizadas em determinados algoritmos, também são liberadas após a conclusão das operações.

2.3 Modularização

A implementação foi organizada em diferentes módulos, com o objetivo de separar responsabilidades e facilitar o desenvolvimento, a manutenção e os testes das funcionalidades.

A comunicação entre os módulos ocorre principalmente por meio da passagem de vetores, quantidades de elementos e parâmetros adicionais às funções. Em operações de cadastro, os contadores são passados por referência, permitindo atualizar a quantidade de registros disponíveis fora do escopo da função.

Essa organização reduz o acoplamento entre funcionalidades e permite que algoritmos individuais sejam testados e modificados sem exigir alterações significativas nos demais componentes do sistema.

2.4 Cadastro e Validação de Dados

O processo de cadastro foi implementado por meio de funções específicas para cada entidade, responsáveis por solicitar, validar e armazenar os dados informados pelo usuário.

Antes de iniciar um cadastro, o sistema verifica se a capacidade máxima do respectivo vetor foi atingida. Caso não exista espaço disponível, a operação é interrompida e o usuário recebe uma mensagem informativa.

Para evitar a inserção de registros parcialmente preenchidos, as funções utilizam inicialmente uma variável temporária da estrutura correspondente. Os campos são preenchidos e validados nessa variável, sendo o registro transferido para o vetor principal somente após a conclusão das etapas de cadastro. O contador de registros é incrementado apenas após essa inserção.

As validações numéricas são específicas para cada entidade. No cadastro de pessoas, a idade deve estar compreendida entre 0 e 120 anos. Para ocorrências, o nível de gravidade assume valores de 1 a 4, enquanto o ano deve estar compreendido entre 1819 (Ano da Fundação de Guarapuava) e o ano corrente. No cadastro de evidências, a relevância deve assumir um valor inteiro entre 1 e 5.

Além da verificação desses intervalos, o sistema trata falhas na leitura de entradas numéricas e utiliza operações de leitura limitadas à capacidade dos vetores de caracteres, evitando a escrita além dos limites dos campos.

O cadastro de evidências possui validações adicionais relacionadas à integridade das associações entre registros. Inicialmente, verifica-se se o identificador da ocorrência informada corresponde a uma ocorrência cadastrada. Caso contrário, o cadastro é cancelado.

Quando uma evidência possui uma evidência-pai, também é verificado se o identificador informado corresponde a um registro existente e associado à mesma ocorrência. Como somente evidências cadastradas anteriormente podem ser utilizadas como referência, não é possível estabelecer relações cíclicas por meio do procedimento de cadastro implementado.

Essas restrições permitem manter as evidências organizadas em estruturas hierárquicas, fornecendo a base necessária para os algoritmos recursivos apresentados na Seção 5. A combinação das validações de entrada, do controle de capacidade e da verificação das relações entre registros contribui para a consistência das informações armazenadas e para o funcionamento das demais operações do sistema.

3 Algoritmos de Ordenação

O sistema implementa dois algoritmos de ordenação: `InsertionSort` e `MergeSort`. Ambos foram desenvolvidos e adaptados para ordenar registros por diferentes atributos, como identificador, idade, nome, gravidade e ano.

3.1 InsertionSort

O Insertion Sort é um algoritmo que constrói uma sequência ordenada progressivamente. A cada iteração, um elemento é selecionado e comparado com os anteriores, deslocando aqueles que forem maiores até encontrar sua posição correta.

Sua complexidade temporal é $\mathcal{O}(n)$ no melhor caso, quando os elementos já estão ordenados, e $\mathcal{O}(n^2)$ nos casos médio e pior. O algoritmo utiliza espaço auxiliar constante ($\mathcal{O}(1)$).

Devido ao seu funcionamento, apresenta bom desempenho em conjuntos pequenos ou parcialmente ordenados, mas torna-se menos eficiente conforme a quantidade de registros aumenta.

3.2 MergeSort

O Merge Sort utiliza a estratégia de divisão e conquista, dividindo recursivamente o conjunto de elementos em duas partes até obter subconjuntos unitários. Posteriormente, esses subconjuntos são intercalados de maneira ordenada.
Sua complexidade temporal é $\mathcal{O}(n \log n)$ nos casos melhor, médio e pior, enquanto a complexidade espacial auxiliar é $\mathcal{O}(n)$, devido à necessidade de um vetor temporário durante a intercalação.
Essa característica torna o algoritmo especialmente adequado para conjuntos maiores, pois apresenta crescimento de custo computacional inferior ao do Insertion Sort no caso médio e no pior caso.

3.3 Ordenação Indireta

Para evitar a movimentação dos registros completos, ambos os algoritmos foram adaptados para operar sobre vetores de índices, armazenados na estrutura `Ordem`.

Durante a ordenação, os atributos dos registros originais são utilizados nas comparações, mas apenas os índices correspondentes são movimentados. Essa abordagem preserva a ordem física dos registros e reduz a quantidade de dados transferidos durante as operações.

A implementação utiliza templates para permitir a seleção de diferentes atributos. Campos numéricos são comparados diretamente, enquanto campos de texto, armazenados em vetores de caracteres, são comparados por meio da função `strcmp()`.

3.4 Comparação Teórica

Algoritmo	Melhor Caso	Caso médio	Pior caso	Espaço Auxiliar
InsertionSort	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$
MergeSort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$

Embora o Merge Sort apresente maior consumo de memória auxiliar, sua complexidade temporal o torna teoricamente mais adequado para grandes conjuntos de registros. O Insertion Sort, por outro lado, apresenta implementação simples e pode ser eficiente em conjuntos menores ou previamente ordenados.

4 Algoritmos de Busca

O sistema implementa dois algoritmos de busca: a Busca Sequencial e a Busca Binária. Ambos permitem localizar pessoas e ocorrências por seus identificadores, retornando a posição do registro encontrado ou o valor -1 caso ele não exista.

4.1 Busca Sequencial

A Busca Sequencial percorre os elementos de um vetor individualmente, comparando o identificador de cada registro com o valor procurado. A operação termina quando o elemento é encontrado ou quando todos os registros foram analisados.

Sua principal vantagem é a possibilidade de operar sobre conjuntos de dados

independentemente de sua ordenação. Entretanto, sua complexidade temporal é $\mathcal{O}(n)$ nos casos médio e pior, pois o número de comparações cresce proporcionalmente à quantidade de registros. No melhor caso, quando o primeiro elemento corresponde ao valor procurado, a complexidade é $\mathcal{O}(1)$.

4.2 Busca Binária

A Busca Binária é um algoritmo que opera sobre conjuntos previamente ordenados. A cada iteração, o elemento central é comparado com o identificador procurado, permitindo descartar aproximadamente metade do intervalo de busca.

Esse procedimento é repetido até encontrar o registro ou até que o intervalo de busca se torne vazio. Sua complexidade temporal é $\mathcal{O}(\log n)$ nos casos médio e pior, enquanto o melhor caso apresenta complexidade $\mathcal{O}(1)$.

Embora seja mais eficiente para grandes conjuntos de dados, a Busca Binária exige que os registros estejam ordenados pelo atributo utilizado na comparação. Caso contrário, o descarte de intervalos pode eliminar posições que contenham o elemento procurado, comprometendo a busca.

4.3 Estratégias de Busca no Sistema

Na implementação desenvolvida, os registros principais permanecem armazenados na ordem de cadastro, com identificadores atribuídos sequencialmente. As operações de ordenação modificam apenas os vetores auxiliares de índices, preservando a organização original dos registros.

Essa abordagem permite executar a Busca Binária diretamente sobre os vetores principais, que permanecem ordenados por identificador, independentemente da ordenação utilizada para exibição dos dados.

Além disso, a atribuição sequencial de identificadores estabelece uma correspondência entre o ID e a posição do registro, possibilitando acesso direto em $\mathcal{O}(1)$, desde que não ocorram exclusões ou lacunas na sequência.

5 Algoritmos Recursivos

O sistema utiliza recursão para analisar as relações entre evidências associadas a uma ocorrência. Foram implementadas duas funcionalidades: o cálculo da relevância total das evidências e a identificação da cadeia de evidências com maior relevância acumulada.

5.1 Modelagem das Relações entre Evidências

As relações entre evidências são representadas por estruturas hierárquicas, nas quais cada evidência pode possuir uma evidência-pai e múltiplas evidências-filhas. Uma evidência sem associação a outra evidência é considerada uma raiz. Dessa forma, uma ocorrência pode possuir diversas árvores independentes, formando uma floresta.

Para facilitar a navegação, o sistema constrói vetores auxiliares que representam as relações entre pais, filhos e irmãos, sem modificar os registros originais. As associações são limitadas a evidências previamente cadastradas e pertencentes à mesma ocorrência, impedindo a formação de ciclos.

5.2 Cálculo Recursivo da Relevância

A função `somarSubarvore()` calcula a soma das relevâncias de uma evidência e de todos os seus descendentes. O caso base ocorre quando a evidência analisada não possui filhos, retornando apenas sua própria relevância. No caso recursivo, a função percorre as evidências-filhas, somando os resultados das chamadas recursivas à relevância do elemento atual.

Para obter a relevância total de uma ocorrência, o sistema executa essa função para cada raiz, acumulando os resultados das diferentes árvores.

5.3 Identificação da Cadeia de Maior Relevância

A segunda funcionalidade recursiva, implementada pela função `melhorCadeiaSubarvore()`, identifica o caminho entre uma evidência-raiz e uma evidência descendente que apresenta a maior soma de relevâncias.

O caso base ocorre quando a evidência não possui filhos, retornando sua relevância individual. No caso recursivo, são analisados os caminhos de cada filho, selecionando aquele que apresenta a maior relevância acumulada. Durante a execução, os índices correspondentes ao melhor caminho são armazenados em um vetor auxiliar, permitindo posteriormente reconstruir e apresentar a sequência de evidências selecionada.

5.4 Análise de Complexidade

Ambas as funções recursivas percorrem cada evidência e suas relações uma única vez, apresentando complexidade temporal $\mathcal{O}(k)$, sendo k a quantidade de evidências pertencentes à ocorrência analisada.

Entretanto, a construção dos vetores auxiliares exige percorrer o conjunto completo de evidências. Assim, considerando n evidências cadastradas no sistema, a complexidade temporal da análise completa é $\mathcal{O}(n)$.

O espaço auxiliar utilizado pelos vetores é $\mathcal{O}(n)$, enquanto a pilha de chamadas recursivas requer $\mathcal{O}(h)$, sendo h a profundidade máxima das árvores.

6 Metodologia de Avaliação de Desempenho

A avaliação de desempenho foi realizada por meio de testes automatizados, desenvolvidos para comparar os algoritmos de ordenação e busca implementados. Foram analisados o número de comparações, a quantidade de movimentações e o tempo de execução, utilizando conjuntos de dados de diferentes dimensões.

6.1 Ambiente de Testes

Os testes foram desenvolvidos em C++, utilizando o padrão C++20 e o compilador g++, com a opção de otimização -O2 em um laptop equipado com 32GB de memória RAM e um processador AMD Ryzen 5 7535HS, rodando um sistema operacional Linux.

A medição do tempo de execução foi realizada por meio da biblioteca chrono, utilizando o relógio steady_clock. Os resultados foram registrados em microssegundos e exportados para arquivos CSV, permitindo sua posterior análise.

6.2 Geração de Dados

Foram utilizados conjuntos de dados sintéticos contendo 1.000, 5.000, 10.000 e 50.000 registros, com identificadores únicos e sequenciais.

Para os testes de ordenação, os índices dos registros foram embaralhados utilizando uma semente fixa para garantir a reprodutibilidade dos experimentos. Ambos os algoritmos receberam a mesma sequência inicial, assegurando condições equivalentes de comparação. Nos testes de busca, os registros permaneceram ordenados por identificador, permitindo a execução correta tanto da Busca Sequencial quanto da Busca Binária.

6.3 Métricas Utilizadas

Foram consideradas três métricas principais:

- Comparações: quantidade de operações de comparação entre valores realizadas pelos algoritmos.
- Movimentações: número de atribuições efetuadas nos vetores de índices durante a ordenação.
- Tempo de execução: intervalo necessário para executar cada algoritmo, medido em microssegundos.

As comparações e movimentações foram contabilizadas por meio de contadores incorporados às implementações, permitindo analisar seu comportamento independentemente das variações de desempenho do hardware ou compilador.

6.4 Procedimentos Experimentais

Para cada dimensão de entrada, os algoritmos Merge Sort e Insertion Sort foram executados uma vez sobre a mesma permutação de índices. O tempo de preparação dos dados, a alocação de memória auxiliar e a verificação dos resultados não foram incluídos na medição.

Os algoritmos de busca foram avaliados em três cenários: localização do primeiro registro, localização do último registro e busca por um identificador inexistente. Cada cenário foi executado 1.000 vezes, sendo registrados os valores acumulados das métricas.

Após cada execução, os resultados foram verificados para garantir que as opeações retornaram resultados corretos. As medições obtidas foram posteriormente utilizadas para comparar o comportamento experimental dos algoritmos com suas respectivas complexidades teóricas.

7 Resultados e Discussão

Nesta seção, são apresentados e discutidos os resultados obtidos nos testes de desempenho dos algoritmos de ordenação e busca. As medições permitem avaliar o comportamento das implementações diante do aumento da quantidade de registros e comparar os resultados com suas respectivas complexidades teóricas.

7.1 Desempenho dos Algoritmos de Ordenação

Registros	**Algoritmo**	**Comparações**	**Movimentações**	**Tempo** (μs)
1.000	Merge Sort	8.707	19.952	108,53
1.000	Insertion Sort	246.622	246.630	297,59
5.000	Merge Sort	55.232	123.616	616,26
5.000	Insertion Sort	6.304.083	6.304.091	6.237,49
10.000	Merge Sort	120.556	267.232	1078.28
10.000	Insertion Sort	25.106.722	25.106.731	31.122,2
50.000	Merge Sort	718.276	1.568.928	8.560,76
50.000	Insertion Sort	625.228.568	625.228.577	974.133,04

Observa-se que o Merge Sort apresentou desempenho superior em todos os conjuntos avaliados. A diferença tornou-se particularmente expressiva com 50.000 registros, quando o Insertion Sort realizou aproximadamente 870 vezes mais comparações e apresentou tempo de execução cerca de 110 vezes maior.

Esse comportamento é compatível com as complexidades teóricas dos algoritmos, evidenciando o crescimento quadrático do Insertion Sort em conjuntos aleatórios e o crescimento $\mathcal{O}(n \log n)$ do Merge Sort.

7.2 Desempenho dos Algoritmos de Busca

Os algoritmos de busca foram avaliados considerando três situações: identificação do primeiro registro, do último registro e busca por um identificador inexistente. A tabela abaixo mostra os resultados para 50.000 registros.

Cenário	**Algoritmo**	**Comparações por busca**	**Tempo por busca** (μs)
Primeiro	Sequencial	1	0,002
Primeiro	Binário	29	0,021
Ultimo	Sequencial	50.000	74,02
Ultimo	Binário	31	0,018
Ausente	Sequencial	50.000	74,15
Ausente	Binário	32	0,019

A Busca Sequencial apresentou melhor desempenho na localização do primeiro registro, pois necessita realizar apenas uma comparação. Entretanto, ao procurar o último elemento ou um identificador inexistente, foi necessário percorrer todo o conjunto.

7.3 Comparação entre Complexidade Teórica e Experimental

Os resultados obtidos estão de acordo com as tendências previstas pela análise assintótica.

Nos algoritmos de ordenação, o aumento do número de registros provocou crescimento significativamente mais acentuado no Insertion Sort do que no Merge Sort. De maneira semelhante, nas buscas pelo último elemento, a Busca Sequencial apresentou crescimento linear na quantidade de comparações, enquanto a Busca Binária apresentou crescimento logarítmico.

Essas observações demonstram que a escolha do algoritmo exerce influência crescente sobre o desempenho à medida que o volume de dados aumenta.

7.4 Limitações

Embora os resultados demonstrem diferenças expressivas entre os algoritmos, alguns fatores devem ser considerados. Os testes utilizaram dados sintéticos, e cada configuração de ordenação foi cronometrada apenas uma vez, estando sujeita a variações decorrentes do ambiente de execução.

Além disso, os contadores de operações introduzem um custo adicional nas implementações, podendo influenciar os tempos medidos. Portanto, os resultados devem ser interpretados como evidências experimentais do comportamento dos algoritmos, e não como valores absolutos de desempenho.

8 Conclusão

O desenvolvimento deste trabalho permitiu integrar diferentes conceitos de algoritmos e programação por meio da implementação de um sistema de gerenciamento de informações relacionadas a investigações criminais. A aplicação desenvolvida contempla funcionalidades de cadastro, consulta, ordenação e análise de evidências, utilizando estruturas de dados e algoritmos implementados em C++.

8.1 Resultados Alcançados

A comparação entre os algoritmos de ordenação demonstrou a influência da complexidade computacional sobre o desempenho. O Merge Sort apresentou resultados superiores ao Insertion Sort nos conjuntos de dados avaliados, especialmente para quantidades elevadas de registros, confirmando as tendências previstas pela análise teórica.

De maneira semelhante, os testes de busca evidenciaram as vantagens da Busca Binária em conjuntos ordenados, enquanto a Busca Sequencial demonstrou efi-

ciência em situações nas quais o elemento procurado se encontra nas primeiras posições.

8.2 Principais Conclusões

Conclui-se que a escolha adequada de algoritmos e estruturas de dados é fundamental para o desenvolvimento de sistemas eficientes e escaláveis. Os resultados obtidos reforçam a importância de combinar a análise teórica de complexidade com a avaliação experimental, permitindo compreender não apenas o funcionamento dos algoritmos, mas também suas limitações e aplicações práticas.

8.3 Melhorias Possíveis

Como possibilidades de melhorias futuras, destacam-se a implementação de persistência de dados, o aprimoramento dos mecanismos de indexação e a realização de testes de desempenho com conjuntos ainda maiores e diferentes distribuições de dados.